BGéSZC Ganz Ábrahám Két Tanítási Nyelvű Technikum

ZÁRÓDOLGOZAT

Gumiverzum

Készítette: Krammer János
Pálfy Alexander Károly

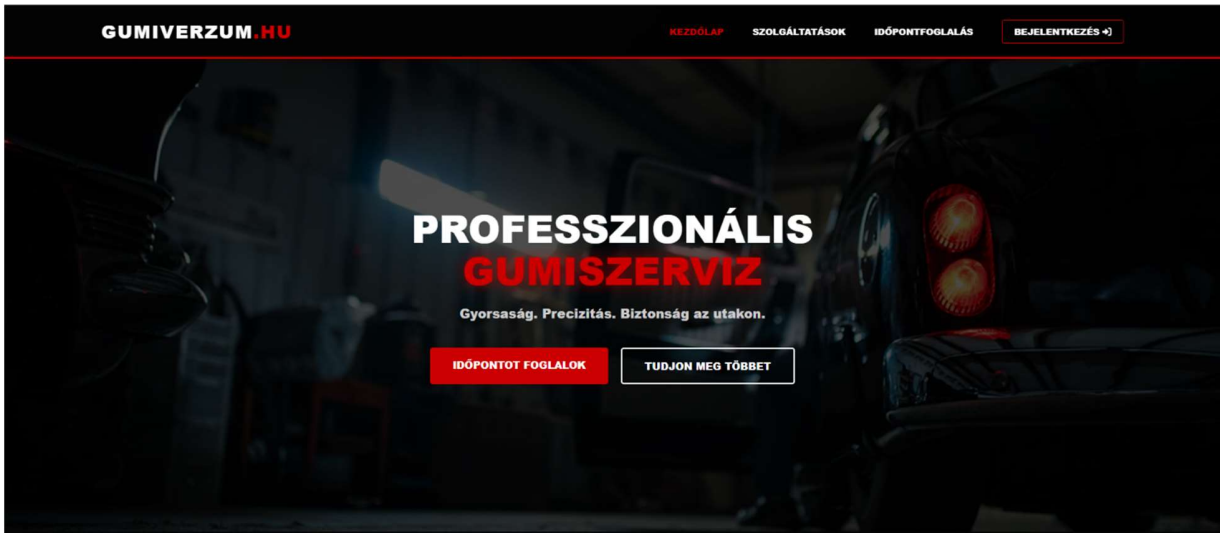
Témavezető: Szalainé Török Edit
Méhes József
Csukárdi Rajmund
Szabó Rózsa

Tartalom

Bevezetés ... 4
Témaismertetés ... 4
Témaválasztás indoklása ... 4
A program funkciói ... 5
Célközönség ... 5
Szakmai célkitűzések ... 6
Telepítési és Indítási Útmutató ... 7
Előfeltételek (Miket kell telepíteni előzetesen?) ... 7
Az Adatbázis inicializálása ... 7
A Backend (Szerver) elindítása ... 7
A Frontend (Kliens) elindítása ... 8
Fejlesztői dokumentáció ... 9
Alkalmazás áttekintés ... 9
Felhasznált szoftverek és fejlesztői környezet ... 9
Felhasznált programozási nyelvek és keretrendszerek ... 9
Backend: Node.js és Express.js ... 9
Frontend: Next.js (React) és Tailwind CSS ... 10
Adatbázis: Microsoft SQL Server ... 10
Adatbázis szerkezete és tervezési elvek ... 11
Táblák ismertetése és Az Adatbázis Elemzése ... 11
A backend működési logikája ... 15
Hitelesítési folyamat és a JWT generálása ... 15
Szabad időpontok dinamikus kalkulációja ... 16
Kétszintű dupla-foglalás gátló algoritmus (Concurrency Control) ... 16
"Upsert" logika az ügyfelek rögzítésénél ... 16
A Frontend architektúra és működési logikája ... 17
Az App Router felépítése és oldalak (Pages) ... 18
Optimista Felület-Frissítés (Optimistic UI Update) komponens-szinten ... 18
Kliensoldali Validáció (Adatellenőrzés) ... 19
Felhasználói Dokumentáció ... 20
Mi is ez a program pontosan, és mire jó? ... 20
Mire lesz szüksége a használathoz? ... 20
A legelső lépés a látogatóknak: A Kezdőlap és Árlista ... 20
Az Időpontfoglalás világa – Lépésről lépésre ... 23
Az Adminisztrátori Vezérlőpult (Dashboard) – A Motorháztető alatt ... 26

Ha hiba történik... (Hibajelzések) 28
Összefoglalás és Köszönetnyilvánítás 29
Projektértékelés és Szakmai tapasztalatok 29
A Gumiverzum utóélete, hasznosulása és Fejlesztési lehetőségek 29
Köszönetnyilvánítás 30
Irodalomjegyzék 31

Bevezetés



Témaismertetés

A Gumiverzum egy modern webalkalmazás, amely asztali számítógépeken és mobiltelefonokon egyaránt kényelmesen használható. Fő célja, hogy összekösse a gépjármű- és gumiszerviz szolgáltatást (kerékcsere, centrírozás, defektjavítás, gumihotel) nyújtó vállalkozást a szolgáltatást igénylő ügyfelekkel.

A szoftver működésének alapelve rendkívül egyszerű a felhasználó számára, ám a háttérben komoly logikai rendszerek biztosítják a zökkenőmentes folyamatokat. Az ügyfelek a rendszeren keresztül gyorsan, átláthatóan tájékozódhatnak az árakról, az elérhető gumiabroncsokról, és egy interaktív naptár segítségével azonnal, várakozás nélkül foglalhatnak szabad időpontot a műhelybe.

Témaválasztás indoklása

A témaválasztás oka elsősorban abból a mindennapi problémából fakad, hogy a legtöbb kisebb-közepes autószerviz és gumiszerviz jelenleg sem rendelkezik korszerű, webalapú időpontfoglaló rendszerrel. Az ügyfeleknek a szervizeket a sűrű nyitvatartási időben, telefonon kell felkeresniük. Ez mind a vállalkozás (munka közbeni megszakítások, olajos kézzel felvett telefonok), mind az ügyfél szempontjából (foglalt jelzés, várakozás a vonalban) időpazarló és frusztráló. Egy központi, 0-24 órában elérhető webalkalmazás létrehozásával ezt a folyamatot egyszerűbbé és automatizálttá lehet tenni.

Szoftverfejlesztői szempontból a projekt hatalmas kihívást és kiváló tanulási lehetőséget jelentett. Egy ilyen platform megtervezése során nem elegendő egy egyszerű adatbázist kezelni. Komplex relációs struktúrát kellett kialakítani, meg kellett valósítani a felhasználók hitelesítését, és – ami a legfontosabb – kezelni kellett a "versenyhelyzeteket, hogy két felhasználó véletlenül se tudjon ugyanarra a percre időpontot foglalni. Ezt egy modern, aszinkron kliensoldali renderelést használó felülettel (Next.js) és backend API-val ötvöztük.

A program funkciói

A Gumiverzum legfontosabb funkciói és moduljai, amelyek a rendszer gerincét alkotják:

- **Dinamikus Szolgáltatás- és Árlista:** A látogatók egy letisztult felületen tekinthetik meg a szerviz által kínált szolgáltatásokat, melyek árai (járműtípusonként bontva) közvetlenül a szerverről, valós időben töltődnek be, így az adminisztrátor ármódosítása azonnal élesedik a felületen.
- **Gumiabroncs Kereső (Kínálat):** Az elérhető gumiabroncsok (nyári, téli, négyévszakos) böngészése márka és méret alapján.
- **Gumiállapot Kalkulátor:** Egy interaktív, beépített modul, amely a felhasználó által megadott DOT-szám (gyártási év) és profilmélység alapján matematikai logikával azonnali javaslatot tesz arra, hogy a gumiabroncs biztonságos-e, vagy cserére szorul.
- **Valós idejű Időpontfoglaló Rendszer:** A platform legfőbb eleme. A felhasználó kiválaszt egy dátumot, majd a rendszer másodpercek alatt lekérdezi az adatbázisból az adott napra vonatkozó foglaltságot, és egy vizuális gomb-rácson (08:00–16:00) megjeleníti a szabad (kattintható) és foglalt (letiltott) időpontokat.
- **Beépített Dupla-foglalás Védelem:** A backend és az adatbázis réteg szigorú szabályokkal akadályozza meg, hogy két kliens egyszerre foglalja le ugyanazt az időpontot.
- **Adminisztrátori Vezérlőpult (Dashboard):** A szerviz üzemeltetői számára fenntartott, jelszóval védett felület. Itt táblázatos formában, statisztikákkal kiegészítve láthatják a beérkező munkákat, és egy gombnyomással "Elvégezve" státuszba helyezhetik azokat.
- **Hitelesítési Rendszer:** JWT (JSON Web Token) alapú beléptetés biztosítja, hogy illetéktelenek ne férhessenek hozzá a megrendelők adataihoz.

Célközönség

A Gumiverzum platformot azzal a céllal hoztuk létre, hogy modern megoldást kínáljunk mindazoknak, akik számára a biztonságos közlekedés alapfeltétel, de nem szeretnének órákat tölteni a szervizek hívogatásával.

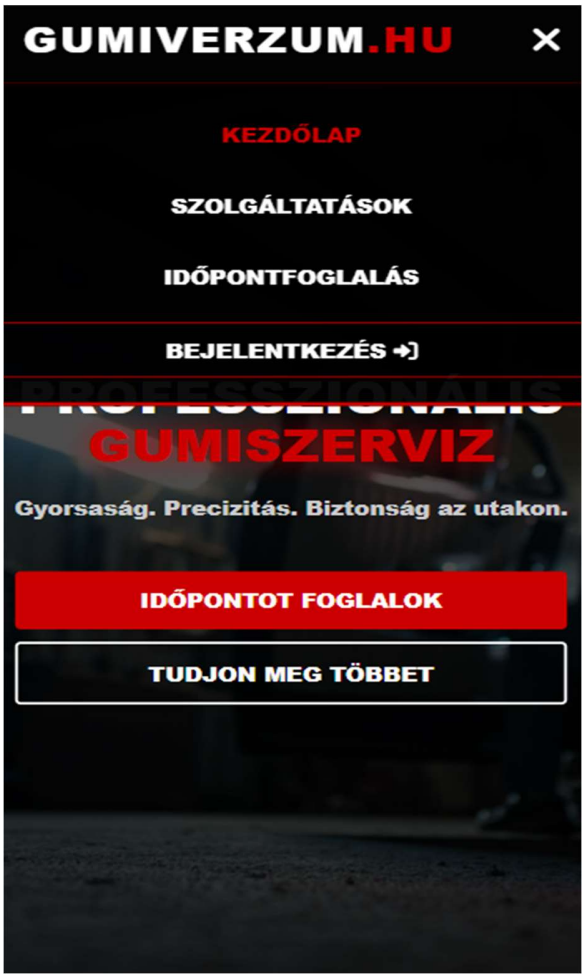
A célközönség kettős:

1. **A gépjármű-tulajdonosok (Ügyfelek):** Akik a szezonális gumicserék idején (tavasz/ősz) gyors, megbízható és azonnali visszaigazolást nyújtó felületet keresnek a foglaláshoz. Számukra a rendszer használata teljesen nyitott, nem igényel regisztrációt, ezzel is növelve a konverziós arányt (foglalási hajlandóságot).
2. **A szerviz tulajdonosai és munkatársai:** Akik a "kockás füzet" helyett egy digitalizált, átlátható napi beosztást szeretnének látni a monitoron vagy a műhelyben elhelyezett tableten, minimalizálva az adminisztrációval töltött időt.

Szakmai célkitűzések

A szoftver fejlesztése során szigorú, modern iparági sztenderdeknek megfelelő szakmai célkitűzéseket fogalmaztunk meg:

- **Rétegelt architektúra alkalmazása:** Alapvető cél volt a frontend (felhasználói felület) és a backend (kiszolgáló és üzleti logika) éles fizikai és logikai szétválasztása. Ez biztosítja a rendszer skálázhatóságát.
- **Korszerű technológia használata:** A frontend oldalon a Next.js (React alapokon) alkalmazása a villámgyors betöltés és az optimalizált felhasználói élmény érdekében. A backend oldalon a Node.js és az Express.js keretrendszer választása a gyors, aszinkron RESTful API kialakításához.
- **Adatbiztonság és hitelesítés:** Modern hitelesítési mechanizmusok (JWT) bevezetése a végpontok védelméhez, és a jelszavak biztonságos kezeléséhez.
- **Robusztus Adatbázis tervezés:** A Microsoft SQL Server relációs adatbázis-kezelőben egy olyan normalizált (3. normálforma) séma kialakítása, amely redundanciamentes adattárolást tesz lehetővé, és hardveres szintű kényszerfeltételekkel védi az adatokat.
- **Reszponzivitás:** A Tailwind CSS keretrendszer segítségével egy olyan felület megalkotása, amely a legkisebb mobiltelefontól a legnagyobb monitorokig hibátlanul jelenik meg.



Telepítési és Indítási Útmutató

Ez a rövid útmutató lépésről lépésre bemutatja, hogyan lehet elindítani a Gumiverzum alkalmazás szerveroldali (Backend) és kliensoldali (Frontend) részét fejlesztői környezetben.

Előfeltételek (Miket kell telepíteni előzetesen?)

Mielőtt belevágna, ellenőrizze, hogy a következő szoftverek telepítve vannak-e a gépén:

1. **Node.js (és npm):** Ez futtatja mind a backendet, mind a frontendet. (Letölthető a nodejs.org oldalról, az LTS verzió javasolt).
2. **Microsoft SQL Server (Express):** Az adatbázis futtatásához.
3. **SQL Server Management Studio (SSMS):** Az adatbázis kezeléséhez és a szkriptek futtatásához.
4. **Fejlesztői kódszerkesztő:** Például Visual Studio Code.

Az Adatbázis inicializálása

1. Nyissa meg az SSMS programot, és csatlakozzon a helyi SQL szerveréhez (pl. localhost\SQLEXPRESS).
2. Futtassa a projekthez mellékelt database.sql fájlt. Ez létrehozza a GumiverzumDB adatbázist, az összes táblát és feltölti a tesztadatokat (árak, szolgáltatások, admin felhasználó).

A Backend (Szerver) elindítása

A backend egy Node.js (Express) projekt, amely a hálózati kéréseket szolgálja ki.

1. Nyisson meg egy parancssort (Terminal).
2. Navigáljon a backend projekt mappájába: cd gumiverzum/backend
3. Hozzon létre egy .env fájlt a mappában az SQL Server hozzáférési adataival (pl. DB_USER, DB_PASS, JWT_SECRET).
4. Telepítse a csomagokat: npm install
5. Indítsa el a szervert: npm run start vagy node server.js
   Ha sikeres, a terminál kiírja: *API fut a http://localhost:5000 porton.*

```
● PS C:\Users\user\OneDrive\Desktop\gumiverzum> cd .\backend\
○ PS C:\Users\user\OneDrive\Desktop\gumiverzum\backend> node .\server.js
  ◇ injected env (5) from .env // tip: ⌁ auth for agents [www.vestauth.com]
  Backend API fut a http://localhost:5012 címen
```

A Frontend (Kliens) elindítása

A frontend egy Next.js alkalmazás. Mivel a frontend API hívásokat intéz a backend felé, mindig ügyeljen rá, hogy a backend már fusson!

1. Nyisson meg egy **új** parancssort (ne zárja be a backendet!).
2. Navigáljon a frontend mappájába: cd gumiverzum/frontend
3. Töltse le a függőségeket: npm install
4. Indítsa el a fejlesztői szervert: npm run dev
   Ha sikeresen elindult, nyissa meg a böngészőjét, gépelje be a http://localhost:3000 címet, és máris láthatja a futó alkalmazást!

```
PS C:\Users\user\OneDrive\Desktop\gumiverzum> cd .\gumiszerviz\
PS C:\Users\user\OneDrive\Desktop\gumiverzum\gumiszerviz> npm run dev

> gumiszerviz@0.1.0 dev
> next dev

▲ Next.js 16.2.3 (Turbopack)
- Local:         http://localhost:3000
- Network:       http://192.168.1.70:3000
✓ Ready in 741ms
```

Fejlesztői dokumentáció

Alkalmazás áttekintés

A Gumiverzum alkalmazás egy modern, többrétegű architektúrára épülő webes rendszer. A fejlesztés során a kliens-oldal és a szerver-oldal éles szétválasztására törekedtünk, amely biztosítja a rendszer méretezhetőségét. Az alkalmazás architektúrája a Single Page Application (SPA) elveit ötvözi a Next.js által nyújtott szerveroldali renderelési lehetőségekkel. Ez a kialakítás lehetővé teszi, hogy a felület villámgyorsan frissüljön, jelentősen növelve a reakciósebességet.

Az alkalmazás kommunikációja REST API hívásokon keresztül történik: a Next.js kliens HTTP kéréseket küld a Node.js szerver felé, amely feldolgozza azokat, elvégzi az SQL adatbázis-műveleteket, majd JSON formátumban visszaküldi az eredményt.

Felhasznált szoftverek és fejlesztői környezet

A fejlesztéshez az iparági sztenderdeknek megfelelő, modern eszközöket választottunk:

- **Visual Studio Code (VS Code):** A teljes kódbázis (frontend és backend) megírásához használt, kódszerkesztő. A beépített integrált terminál hatalmas segítséget nyújtott a párhuzamos Node.js folyamatok futtatásában.
- **Postman:** A backend fejlesztése során az API végpontok tesztelésére használt szoftver. Ennek segítségével a frontend megépítése előtt is ellenőrizhető volt a token-generálás és a foglalási logika.
- **SQL Server Management Studio (SSMS) 20.x:** Az adatbázis vizuális megtervezésére, a lekérdezések (Query-k) tesztelésére és a táblák kapcsolatainak ellenőrzésére használt hivatalos Microsoft eszköz.

Felhasznált programozási nyelvek és keretrendszerek

Backend: Node.js és Express.js

Az üzleti logika megvalósításához a **JavaScript (Node.js)** futtatókörnyezetet választottuk. Ennek óriási előnye, hogy a teljes Full-Stack projekt (kliens és szerver) egyetlen programozási nyelven íródhatott, csökkentve a kontextusváltásból adódó mentális terhet.

- **Express.js:** Ez a minimalista, de rendkívül gyors webes keretrendszer felel a bejövő HTTP kérések fogadásáért és a végpontok kiajánlásáért. Kezeli a CORS beállításokat, hogy a frontend zavartalanul kommunikálhasson a szerverrel, és köztes szoftvereken keresztül validálja a bejövő JSON csomagokat.
- **mssql (npm csomag):** A Node.js hivatalos adatbázis-kliense a Microsoft SQL Serverhez. Segítségével paraméterezett, aszinkron (async/await) lekérdezéseket futtathatunk, ami megvéd a hírhedt SQL Injection támadások ellen.

Frontend: Next.js (React) és Tailwind CSS

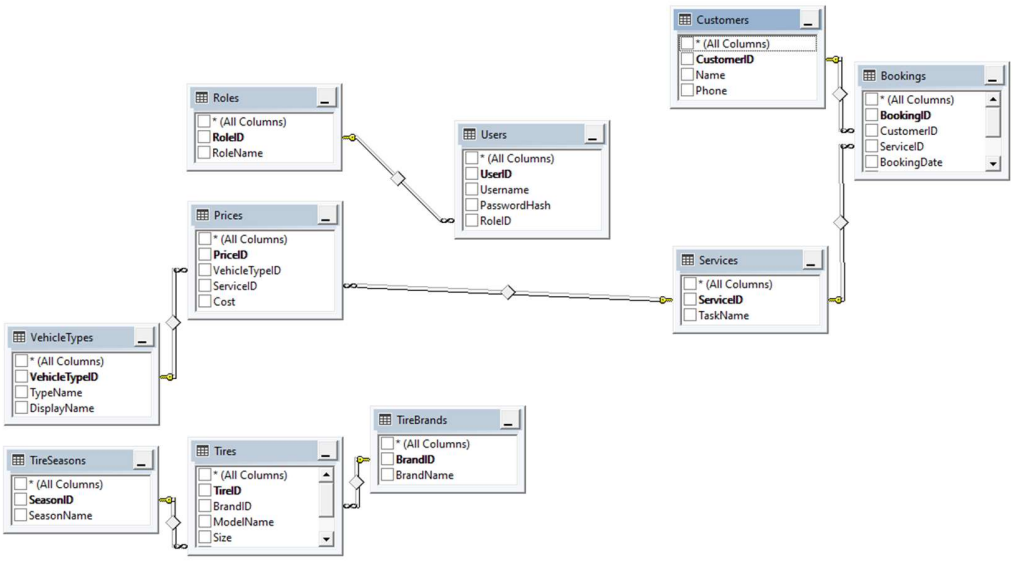
A felhasználói felület megalkotásához a React könyvtárra épülő **Next.js** keretrendszert alkalmaztuk.

- **Next.js (App Router):** Azért választottuk, mert a fájlrendszer-alapú útválasztása (routing) rendkívül átlátható projektstruktúrát eredményez. A komponens-alapú felépítés lehetővé teszi a kód újrahasznosítását (pl. egyedi gombok, kártyák, navigációs sávok).
- **Tailwind CSS:** Egy Utility-first CSS keretrendszer. Hagyományos, több ezer soros CSS fájlok írása helyett a stílusokat közvetlenül a HTML (JSX) elemek osztályaiba (class) írjuk be. Segítségével elképesztő sebességgel lehetett reszponzív, mobilbarát kinézetet létrehozni (pl. md:flex, sm:w-full osztályok használatával).

Adatbázis: Microsoft SQL Server

A relációs adatbázis-kezelő (RDBMS) elengedhetetlen egy foglalási rendszernél. Míg a modern NoSQL (pl. MongoDB) adatbázisok rugalmasak, nem biztosítják azt a szigorú adatintegritást és tranzakció-kezelést (ACID tulajdonságok), ami a kettős foglalások elkerüléséhez szükséges. Az MSSQL beépített UNIQUE CONSTRAINT és idegen kulcs (Foreign Key) rendszere hardveres szinten garantálja, hogy a rendszer ne omolhasson össze adathibák miatt.

Adatbázis szerkezete és tervezési elvek



A GumiverzumDB adatbázis a 3. normálformának (3NF) megfelelően, szigorú tervezési elvek alapján készült. A redundancia (adattöbbszöröződés) elkerülése, valamint a későbbi skálázhatóság volt a fő szempont. A rendszer összesen 10 táblából áll. Az alábbiakban minden egyes táblát, annak mezőit, a mezők típusát és a kapcsolatok miértjét részletezzük.

Táblák ismertetése és Az Adatbázis Elemzése

1. Roles (Szerepkörök) tábla

A jogosultságkezelés alapja. Egy egyszerű szótártábla.

RoleID (int, IDENTITY, PRIMARY KEY): Egyedi sorszám.

RoleName (nvarchar(50), UNIQUE): A szerepkör megnevezése. Jelenleg 'Admin' és 'Operator' szintek támogatottak. Ez a felépítés garantálja, hogy a jövőben új szintek adhatóak hozzá a kód átírása nélkül.

2. Users (Felhasználók) tábla

Az adminisztrátori és dolgozói hozzáféréseket tárolja. (A publikus vásárlók nem ide kerülnek!)

UserID (int, IDENTITY, PRIMARY KEY): A felhasználó egyedi azonosítója.

Username (nvarchar(50), UNIQUE): Bejelentkezési név (pl. "admin").

PasswordHash (nvarchar(MAX)): A jelszavak tárolására. Szigorú biztonsági elv, hogy nyílt jelszót sosem tárolunk, ide csak a titkosított formátum kerülhet.

RoleID (int, FOREIGN KEY): Idegen kulcs, amely összeköti a fiókot a Roles táblával.

3. **VehicleTypes (Járműtípusok) tábla**

Mivel egy gumiszerviz árazása drasztikusan eltér egy személyautó és egy teherautó esetén, a rendszert erre fel kellett készíteni.

VehicleTypeID (int, IDENTITY, PRIMARY KEY): Egyedi azonosító.

TypeName (nvarchar(50), UNIQUE): Belső technikai azonosító (pl. "szemely", "teher").

DisplayName (nvarchar(100)): A frontend felületén, a legördülő menükben megjelenő barátságos szöveg (pl. "Személyautó").

5. **Services (Szolgáltatások) tábla**

Az elvégezhető munkák katalógusa.

ServiceID (int, IDENTITY, PRIMARY KEY): Egyedi azonosító.

TaskName (nvarchar(100), UNIQUE): A szolgáltatás neve (pl. "Szerelés + Centrírozás", "Defektjavítás", "Gumihotel").

5. Prices (Árak) tábla - Kapcsolótábla

A relációs adatbázisok egyik legszebb megoldása. Mivel egy szolgáltatásnak (pl. Kerékcsere) más az ára személy- és teherautó esetén, ez a tábla oldja fel a több-a-többhöz (N:M) kapcsolatot.

PriceID (int, IDENTITY, PRIMARY KEY): Egyedi azonosító.

VehicleTypeID (int, FOREIGN KEY): Melyik járműre vonatkozik (Mutat a VehicleTypes táblára).

ServiceID (int, FOREIGN KEY): Melyik szolgáltatásra vonatkozik (Mutat a Services táblára).

Cost (int): A konkrét bruttó ár forintban. Így az árváltoztatás csak egyetlen cella átírását jelenti, anélkül, hogy a struktúra sérülne.

6. TireBrands (Gumiabroncs Márkák) tábla

BrandID (int, IDENTITY, PRIMARY KEY): Egyedi azonosító.

BrandName (nvarchar(50), UNIQUE): Kínált márkák nevei (Michelin, Hankook stb.). Szótártáblaként funkcionál, hogy elkerüljük az elgépeléseket.

7. TireSeasons (Szezonok) tábla

SeasonID (int, IDENTITY, PRIMARY KEY): Egyedi azonosító.

SeasonName (nvarchar(50), UNIQUE): "Nyári", "Téli", "Négyévszakos" kategóriák.

8. **Tires (Gumiabroncsok) tábla**

A szerviz webshop-szerű kínálatát tároló entitás.

TireID (int, IDENTITY, PRIMARY KEY): Egyedi azonosító.

BrandID (int, FOREIGN KEY): Hivatkozás a TireBrands táblára.

SeasonID (int, FOREIGN KEY): Hivatkozás a TireSeasons táblára.

ModelName (nvarchar(100)): A gumi pontos modellje (pl. "Pilot Sport 5").

Size (nvarchar(50)): Fizikai méret (pl. "225/45 R17").

Price (int): A gumiabroncs darabára forintban.

9. Customers (Ügyfelek) tábla

Az online időpontot foglaló magánszemélyek adatai.

CustomerID (int, IDENTITY, PRIMARY KEY): Egyedi azonosító.

Name (nvarchar(150)): Az ügyfél teljes neve.

Phone (nvarchar(20)): Telefonszám. Azért szöveges formátum, mert a nullával (06) kezdődő értékeket a szám formátumok (int) csonkítanák. Ennek segítségével a szerviz vissza tudja hívni az ügyfelet probléma esetén.

10. **Bookings** (Foglalások) tábla A rendszer legfontosabb, tranzakcionális központja. Ide fut be minden foglalás.

BookingID (int, IDENTITY, PRIMARY KEY): Egyedi azonosító.

CustomerID (int, FOREIGN KEY): Ki foglalta az időpontot (Mutat a Customers táblára).

ServiceID (int, FOREIGN KEY): Milyen munkát szeretne elvégeztetni (Mutat a Services táblára).

BookingDate (DATE): A munka pontos naptári napja.

BookingTime (nvarchar(5)): A lefoglalt óra:perc (pl. "09:00", "14:00").

Status (nvarchar(20), DEFAULT 'Pending'): Állapotgép. Alapértelmezetten 'Pending' (Függőben / Aktív). Az adminisztrátor ezt állítja át 'Completed' (Elvégezve) státuszra.

CreatedAt (DATETIME, DEFAULT GETDATE()): A rögzítés mikroszekundum pontos időbélyege a naplózhatóság (Audit) kedvéért.

Kritikus Megszorítás (Constraint): A Bookings táblán egy kritikus CONSTRAINT UQ_BookingSlot UNIQUE (BookingDate, BookingTime) kényszerfeltétel található. Ez a hardveres fék biztosítja, hogy ha két ember egy időben próbál meg a "10:00" órára foglalni április 20-án, az adatbázis szigorúan visszautasítja a második beszúrást (INSERT). Ezáltal a dupla foglalás elméleti szinten is lehetetlenné válik.

A backend működési logikája

A Node.js/Express backend feladata sokkal több, mint puszta adatolvasás. Biztonsági ellenőrzéseket, adatalakításokat és üzleti folyamatokat vezérel.

Hitelesítési folyamat és a JWT generálása

A bejelentkezés egy állapotmentes mechanizmust használ. Amikor az adminisztrátor megadja a nevét és jelszavát, az authController.js fogadja a POST kérést. Lekérdezi az MSSQL Users tábláját. Ha a párosítás egyezik, a backend elindít egy token-generáló algoritmust a jsonwebtoken csomag segítségével. Létrehoz egy digitálisan aláírt adatkapszulát, amely tartalmazza a felhasználó ID-ját, de a jelszavát semmiképp. Ezt a Tokent küldi vissza a Next.js kliensnek. Bármely további kérés érkezik a védett admin végpontokra (pl. foglalások lekérése), a backend egy Middleware (köztes szoftver) segítségével ellenőrzi a token érvényességét, mielőtt beengedné az adatbázis közelébe a kérést.

Szabad időpontok dinamikus kalkulációja

A bookingController.js tartalmazza a legokosabb algoritmust. Amikor a felhasználó a naptárban rábök egy napra, a kliens azonnal küld egy GET kérést: /api/slots?date=2026-04-20.

1. A backend lefuttat egy SQL lekérdezést, amely kilistázza a Bookings táblából az összes olyan BookingTime-ot, ami erre a napra esik.
2. A memóriában előállít egy statikus tömböt a teljes nyitvatartásról: ['08:00', '09:00', '10:00', ..., '16:00'].
3. Egy iteráció (ciklus) segítségével összeveti a foglalt időpontokat a teljes nyitvatartással.
4. Generál egy új objektumtömböt, ahol minden időponthoz rendel egy available: true/false flaget.
5. Ezt a JSON listát küldi vissza a kliensnek, ami alapján a böngésző kiszínezi a gombokat zöldre (szabad) vagy szürkére (foglalt).

Kétszintű dupla-foglalás gátló algoritmus

[Kép helye: Ide szúrd be a dupla foglalás megakadályozását bemutató kommunikációs diagramot]

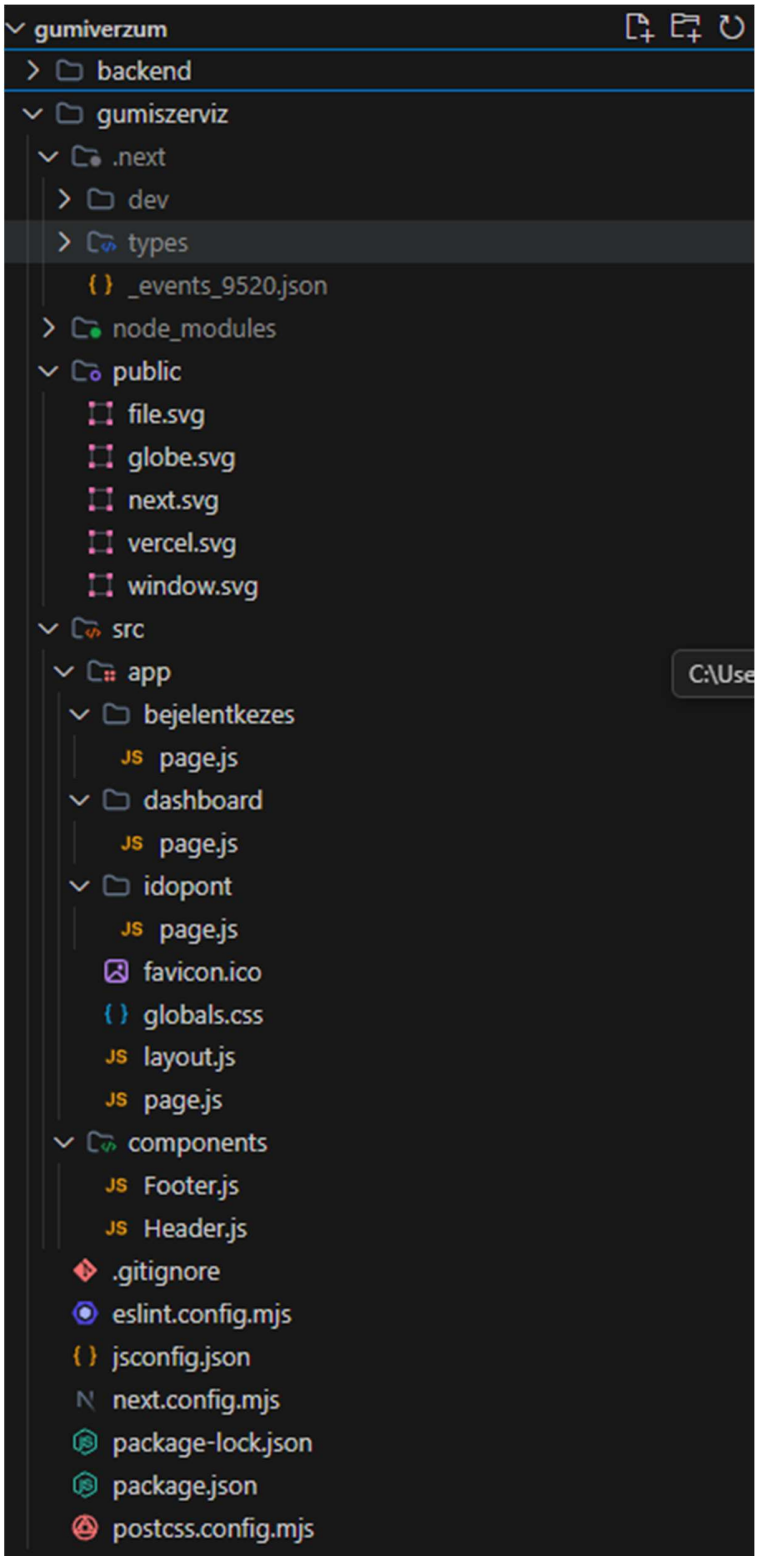
A foglalás véglegesítésekor a rendszer kétszintű védelmet alkalmaz a versenyhelyzetek elkerülésére:

- **Első szint (Alkalmazás/Node.js réteg):** A végpont a foglalás INSERT parancsa előtt futtat egy ellenőrző SELECT lekérdezést. Ha a kiszemelt időpont már szerepel az adatbázisban, a szerver 409-es (Conflict) hibakóddal megszakítja a folyamatot.
- **Második szint (Adatbázis réteg):** Ha egy extrém terheléses esetben két kérés pontosan ugyanazon a milliszekundumon jutna át a Node.js rétegen, az SQL Server a korábban beállított UNIQUE CONSTRAINT megszorítás miatt adatbázis-szintű hibát dob a második lekérdezésre, amelyet a backend catch blokkja lekezel, és értesíti a felhasználót, hogy az időpont időközben betelt.

"Upsert" logika az ügyfelek rögzítésénél

Az ügyfelek telefonszám szerinti azonosítása miatt nem hozunk létre duplikált rekordokat. A backend algoritmusa a foglalás rögzítésekor megvizsgálja: létezik-e már az ügyfél (Phone) a Customers táblában? Ha igen, lekéri annak CustomerID-ját. Ha nem, először végrehajt egy INSERT-et a Customers táblába, majd az újonnan kapott ID-val rögzíti a foglalást a Bookings táblában. Mindez egy gyors folyamatban megy végbe.

A Frontend architektúra és működési logikája



A Gumiverzum frontendje a Next.js (App Router) technológiára épül. Célja a felhasználói felület megszakítások nélküli, azonnali reagálása, és a tökéletes mobil-élmény biztosítása a Tailwind CSS-sel.

Az App Router felépítése és oldalak (Pages)

- **app/layout.jsx:** Az alkalmazás gyökérkomponense. Felelős a globális struktúráért és a Tailwind stílusok betöltéséért. Ebbe a burkolóba kerül beillesztésre a közös navigációs fejléc (Navbar), amely így minden oldalváltásnál a helyén marad anélkül, hogy újra kellene tölteni.
- **app/page.jsx (Kezdőlap):** A landoló oldal. Betöltéskor a komponens lekéri a szerverről a gumiabroncs-kínálatot és az árakat. Itt kapott helyet a *Gumiállapot Kalkulátor* komponens is, amely kliensoldali JavaScript logikával (a React állapotkezelését használva) azonnal számol, hálózati forgalom nélkül.
- **app/foglalas/page.jsx (Időpontfoglaló nézet):** A legösszetettebb kliensoldali logika helye. Ez a fájl kezeli a HTML5 Datepicker-t (dátumválasztó). Tartalmazza a React useState és useEffect hook-okat. Amint a selectedDate állapota megváltozik, a useEffect automatikusan, a háttérben elindít egy API hívást (Fetch) a szabad időpontokért, majd újra-rendereli a felületet az eredményekkel.
- **app/bejelentkezes/page.jsx:** A biztonsági belépési pont az adminisztrátor számára. Tartalmazza a jelszómezőt és a hibaüzenetek (pl. "Hibás jelszó") állapotát. Sikeres belépés esetén a localStorage-ba menti a tokent, majd a next/navigation routerével azonnal átirányít a védett /dashboard oldalra.
- **app/dashboard/page.jsx (Admin Vezérlőpult):** Egy védett útvonal. A komponens a betöltődés pillanatában (onMount) ellenőrzi a token meglétét. Ha nincs token, biztonsági okokból visszadobja a felhasználót a bejelentkezésre. Ha van, aszinkron módon lekéri a "Pending" státuszú foglalásokat, és egy statisztikákkal (összes mai munka, jövőbeli munkák) gazdagított táblázatot rajzol ki.

Optimista Felület-Frissítés (Optimistic UI Update) komponens-szinten

A Dashboardon található az "Elvégezve" gomb. A kiváló felhasználói élmény érdekében itt egy trükköt, a *Optimistic UI Update*-et alkalmaztuk. Amikor az admin rákattint a gombra, a frontend nem várja meg a lassú hálózati kommunikációt: azonnal, a memóriából kitörli a foglalás sorát, így a felület villámgyorsan reagál. Ezzel párhuzamosan elindít egy HTTP PATCH kérést a backend felé. Ha a kérés szerverhibára fut (pl. megszakad a WiFi), a catch ág lefut, megjelenít egy hibaüzenetet, és "visszagörgeti" (rollback) a sort a táblázatba.

Kliensoldali Validáció (Adatellenőrzés)

A felesleges hálózati terhelés elkerülése végett az űrlapok beépített ellenőrzéssel rendelkeznek. A telefonszám beviteli mező egy Reguláris Kifejezés alapján folyamatosan vizsgája az adatot (^06[0-9]{8,9}$). Ha a felhasználó véletlenül betűket vagy "06" helyett mást ír be, a gomb inaktív marad, és egy piros figyelmeztető szöveg jelenik meg közvetlenül a mező alatt, azonnali visszacsatolást adva a hibáról.

Felhasználói Dokumentáció

A szoftverek világa sokszor ijesztőnek és bonyolultnak tűnhet. Amikor megterveztük a Gumiverzum alkalmazást, az elsődleges szempontunk az volt, hogy senkinek ne kelljen informatikai diplomával rendelkeznie ahhoz, hogy időpontot tudjon foglalni a gépjárműve szervizelésére, vagy hogy a műhelyben dolgozók átlássák a napi beosztást.

Ez a fejezet egy rendkívül részletes, minden gombra kiterjedő kézikönyv. Úgy írtuk meg, mintha csak ott ülnénk Ön mellett a képernyő előtt, és lépésről lépésre mutatnánk meg, mit hol talál. Kérjük, dőljön hátra, és olvassa el figyelmesen!

Mi is ez a program pontosan, és mire jó?

Képzelje el a Gumiverzumot úgy, mint egy éjjel-nappal nyitva tartó, digitális recepciót.

Ha **Ön egy autós**, akinek le kell cserélnie a nyári gumit télire, vagy defektet kapott, akkor Ön az "Ügyfél". Az oldalon körülnézhet, láthatja az árakat, ellenőrizheti, hogy a jelenlegi gumija jó-e még, majd **ingyenesen, regisztráció nélkül** foglalhat egy pontos órát magának. Nem kell többet hívogatnia a szervizt, amikor azok épp az autók alatt fekszenek.

Ha **Ön a szerviz dolgozója vagy tulajdonosa**, akkor Önnek ez egy virtuális iroda és határidőnapló. Nem kell többé füzetekben lapozgatni: a képernyőn pontosan, órára lebontva látja, ki jön, milyen autóval, és mit kell csinálni.

Mire lesz szüksége a használathoz?

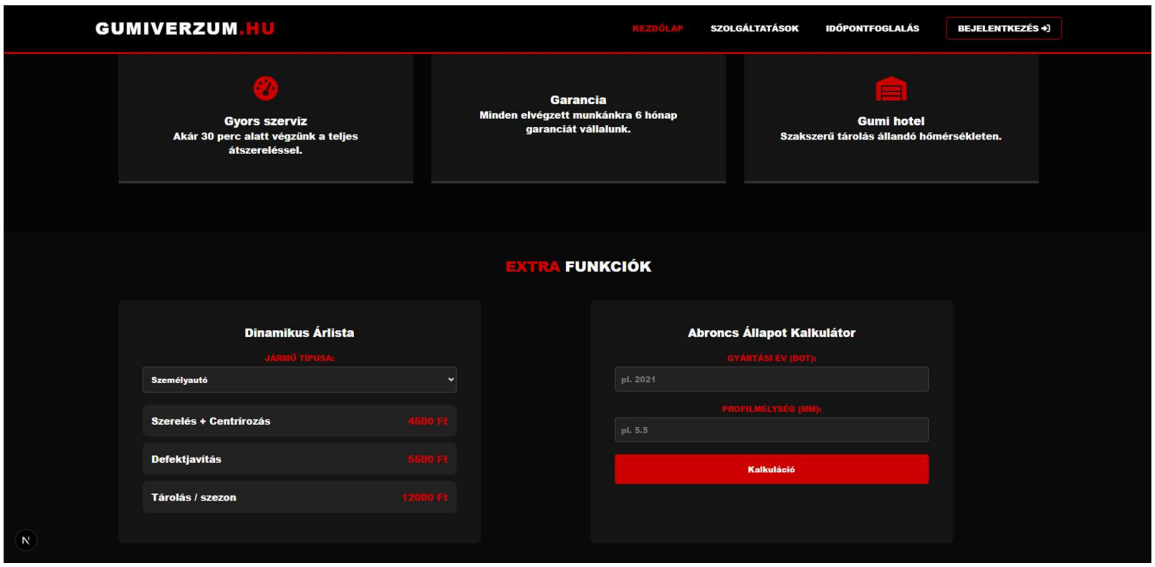
A legjobb hírünk van az Ön számára: a Gumiverzum használatához **semmit nem kell letöltenie vagy telepítenie!** Nincs szükség drága számítógépre.

- **Egy eszköz:** Lehet egy asztali számítógép, egy tablet a műhelyben, vagy akár az a mobiltelefon, ami épp a zsebében van. A program automatikusan "összehúzza" magát, hogy a telefon képernyőjén is tökéletesen olvasható legyen.
- **Internetkapcsolat:** WiFi vagy mobilinternet.
- **Egy webböngésző program:** Google Chrome, Firefox, vagy Safari. Bármelyiket használhatja.

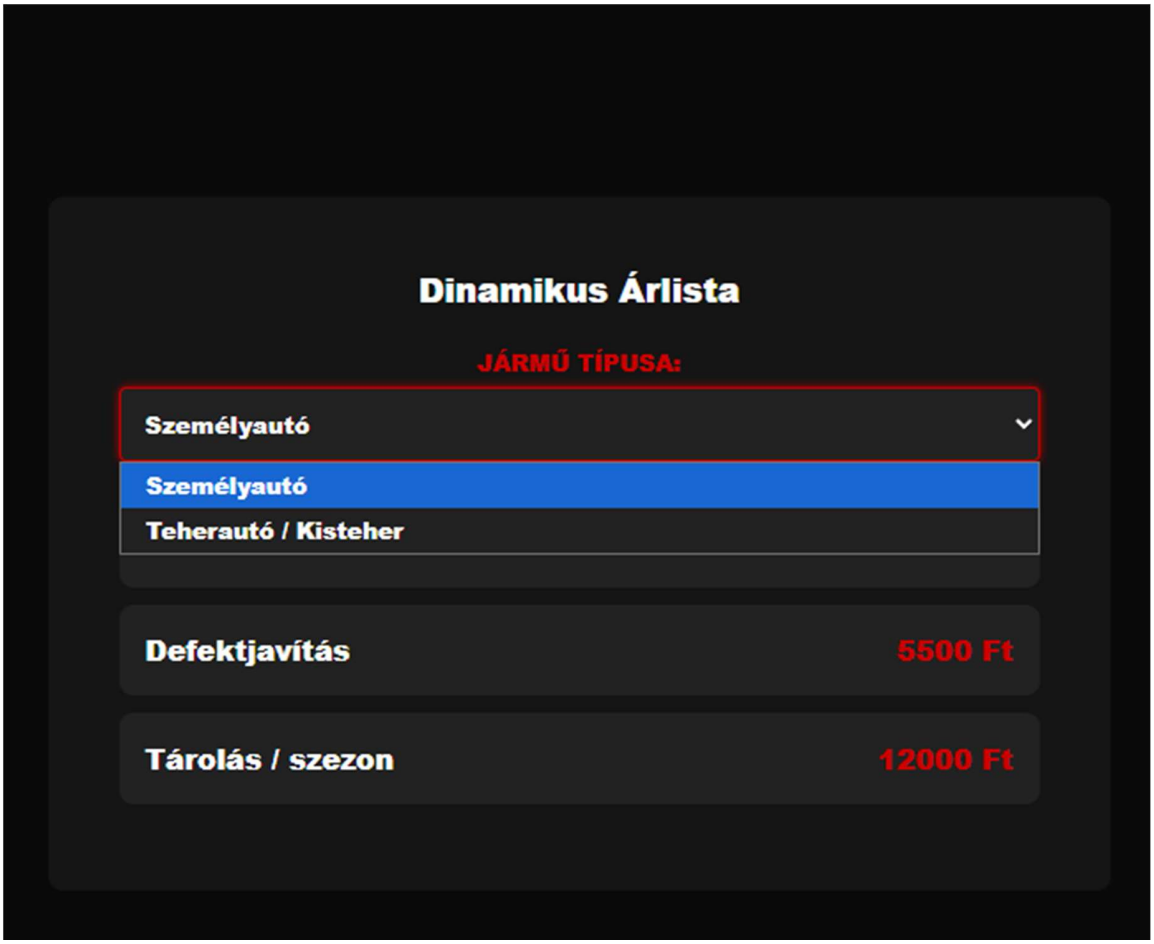
A legelső lépés a látogatóknak: A Kezdőlap és Árlista

Nyissa meg a böngészőjét, és a legfelső hosszú fehér sávba (a címsorba) gépelje be a szerviz webcímét, majd nyomja meg az Entert.

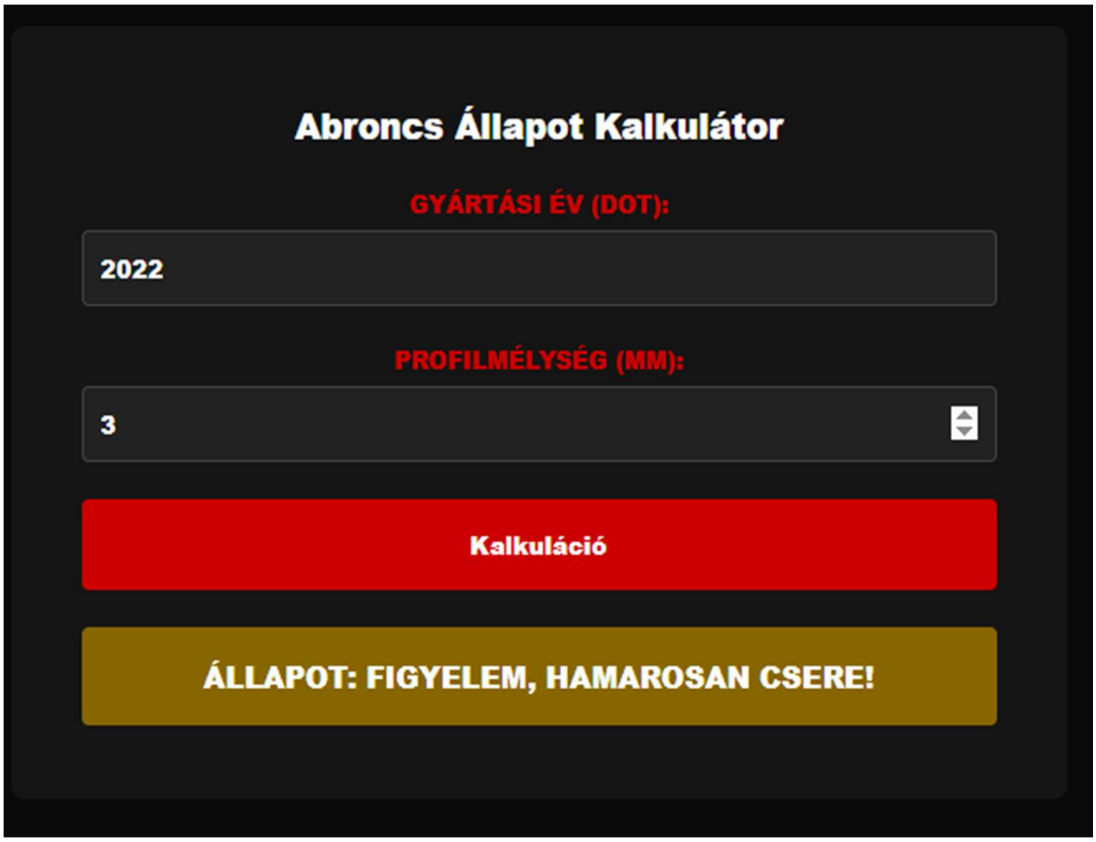
Azonnal a főoldalon találja magát. Itt három fontos dolgot tud csinálni:



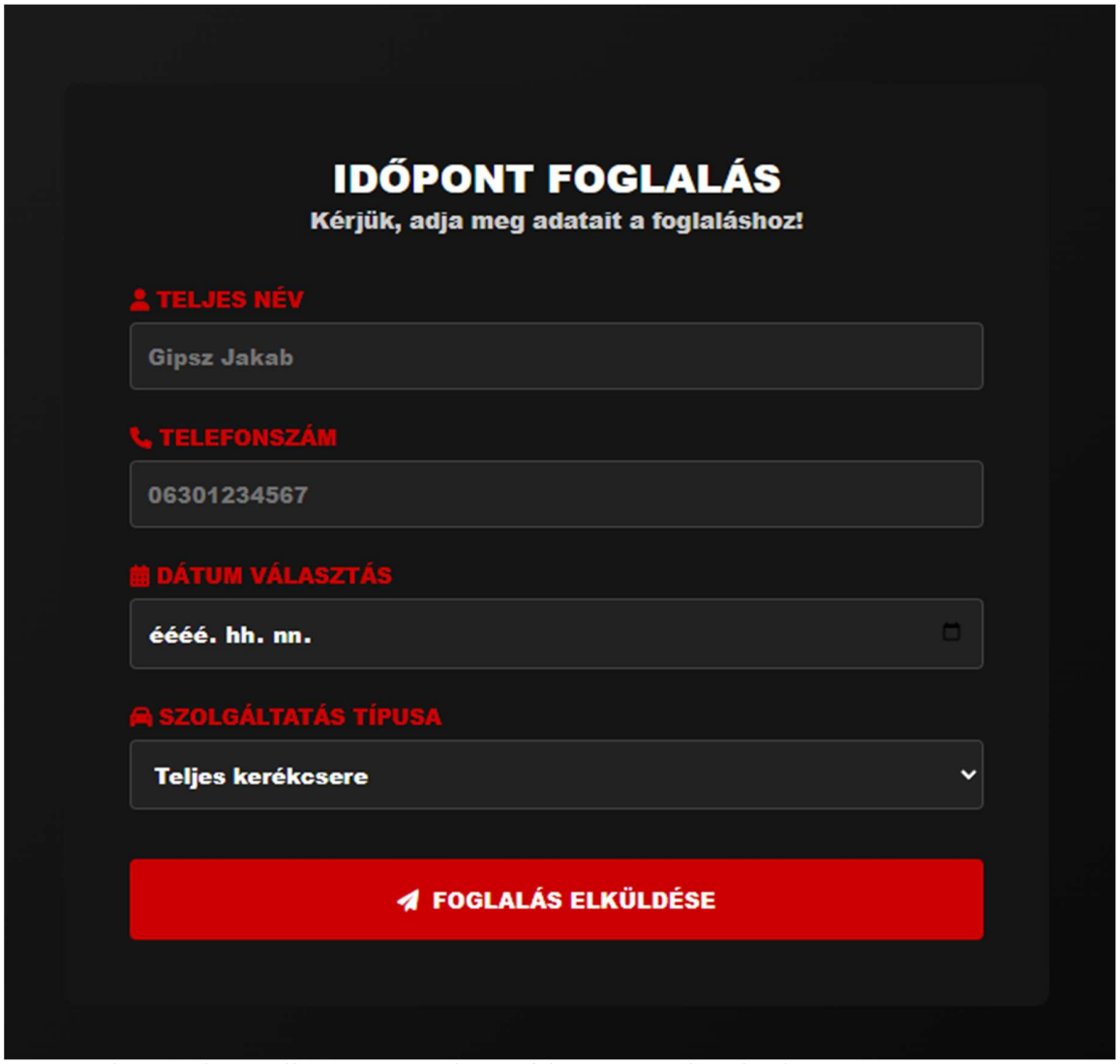
1. **Szolgáltatások böngészése:** Láthatja, hogy a szerviz vállal-e centrírozást vagy gumitárolást (Gumihotelt).
2. **Az Árlista megtekintése:** Egy letisztult táblázatban láthatja a pontos forintosított árakat. Egy kapcsolóval (Gomb) válthat a Személyautós árak és a Teherautós árak között. Így sosem éri meglepetés a kasszánál.



3. **A Gumiállapot Kalkulátor:** Egy nagyszerű kis eszköz! Van benne két csúszka és beviteli mező. Írja be, hogy mikor gyártották a gumit (ezt a gumi oldalán találja meg a DOT szó után), és adja meg, hány milliméter mély a bordázat rajta. A program azonnal kiszámolja, és zöld vagy piros színnel kiírja, hogy Ön biztonságosan elindulhat-e a családi nyaralásra, vagy azonnal cserélni kell az abroncsokat!

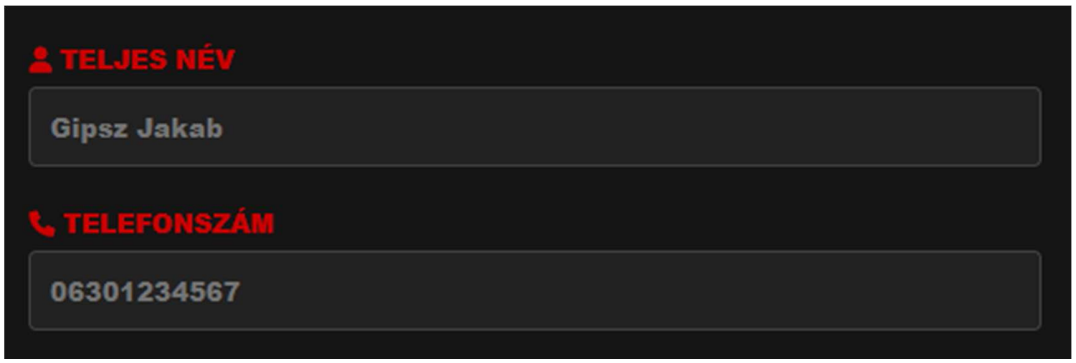


Az Időpontfoglalás világa – Lépésről lépésre



Ha úgy döntött, hogy elhozza az autóját, a felső menüsorban kattintson az **"Időpontfoglalás"** gombra. Ne izguljon, nem kell fiókot regisztrálnia vagy jelszavakat megjegyeznie! A folyamat 4 egyszerű lépésből áll:

1. **Adatok megadása:**
   - **Teljes név:** Gépelje be a nevét (pl. Kovács István).
   - **Telefonszám:** Ide nagyon figyeljen! A rendszer csak a 06 kezdetű, kötőjelek és szóközök nélküli (10-11 számjegyű) telefonszámokat fogadja el (pl. 06301234567). Ha betűket ír be, a gép piros betűkkel figyelmeztetni fogja!



2. **A Szolgáltatás kiválasztása:** Egy legördülő menüből (egy lefelé mutató kis nyíllal) válassza ki, mit szeretne csinálni (pl. Teljes kerékcsere).
3. **A Dátum és az Időpont:**
   - Kattintson a dátum mezőre. Lenyílik egy naptár. (Múltbéli napot nem tud kiválasztani, a gép okos!)
   - Ahogy rábökött egy napra, a naptár alatt **varázslatos módon megjelennek a kis időpont-gombok** (08:00, 09:00, stb.).
   - Amelyik gomb **Zöld** és rá van írva, hogy "Szabad", arra nyugodtan rákattinthat.
   - Amelyik gomb **Szürke**, nem tud rákattintani. Azt valaki más már lefoglalta az Ön orra elől.

4. **A Foglalás véglegesítése:** Ha minden adat jó, és kiválasztotta az órát (a gomb zölden marad), nyomja meg a hatalmas **"FOGLALÁS ELKÜLDÉSE"** gombot a lap alján! A képernyőn egy szép zöld üzenet jelenik meg: *Sikeres foglalás! Várjuk szeretettel.* Kész is van!

IDŐPONT FOGLALÁS

Kérjük, adja meg adatait a foglaláshoz!

TELJES NÉV

Gipsz Jakab

TELEFONSZÁM

06301234567

DÁTUM VÁLASZTÁS

2026. 04. 22.

IDŐPONT VÁLASZTÁS

8:00 Szabad	9:00 Szabad	10:00 Szabad	11:00 Szabad	12:00 Szabad
13:00 Szabad	14:00 Szabad	15:00 Foglalt	16:00 Szabad	

SZOLGÁLTATÁS TÍPUSA

Teljes kerékcsere

FOGLALÁS ELKÜLDÉSE

Sikeres foglalás! 2026-04-22 – 15:00 óra. Hamarosan visszahívjuk.

Az Adminisztrátori Vezérlőpult (Dashboard) – A Motorháztető alatt

Ha Ön a szerviz alkalmazottja vagy a vizsgáztató/tesztelő tanár, akkor most megmutatjuk, hogy néz ki a rendszer a pult túloldaláról.

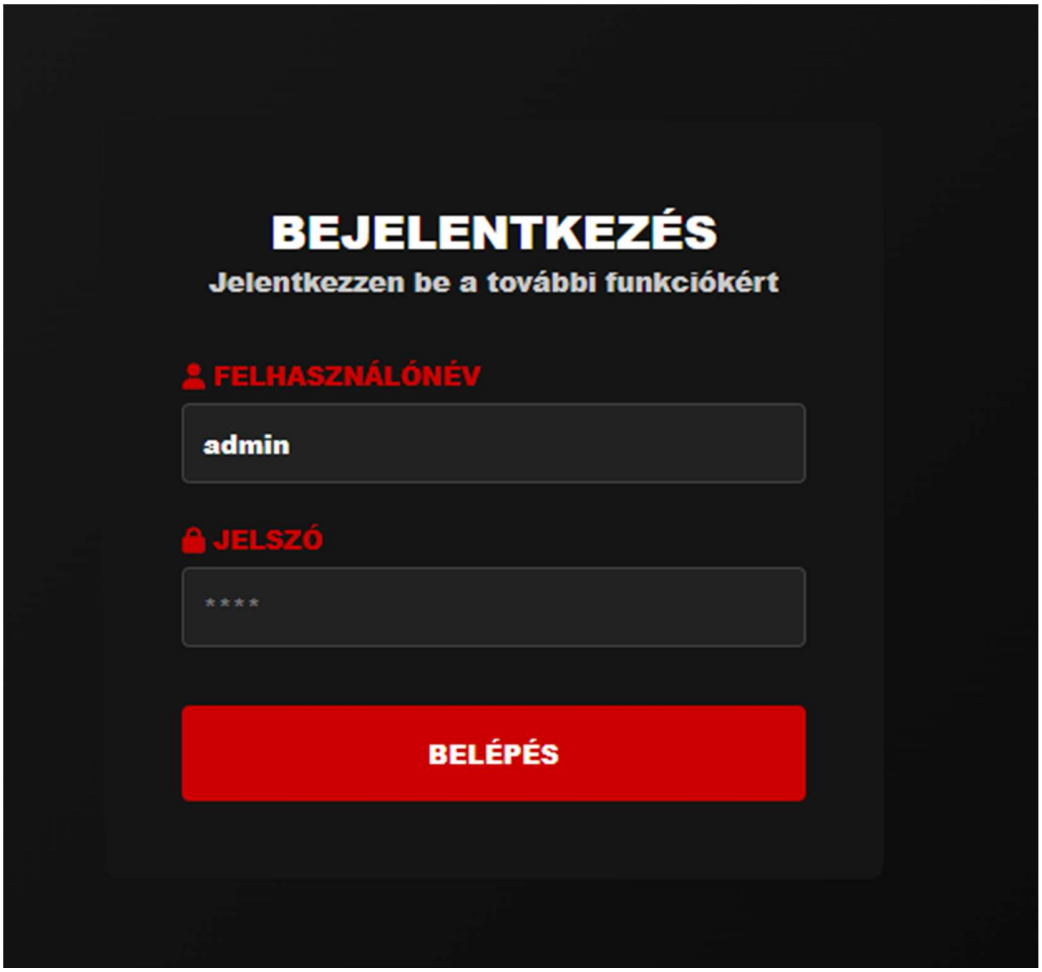
A Bejelentkezés:

A jobb felső sarokban talál egy "BEJELENTKEZÉS" gombot. Kattintson rá.

Itt megjelenik egy biztonsági kapu. A felhasználónév fixen be van írva (admin).

A jelszó mezőbe írja be: 1234

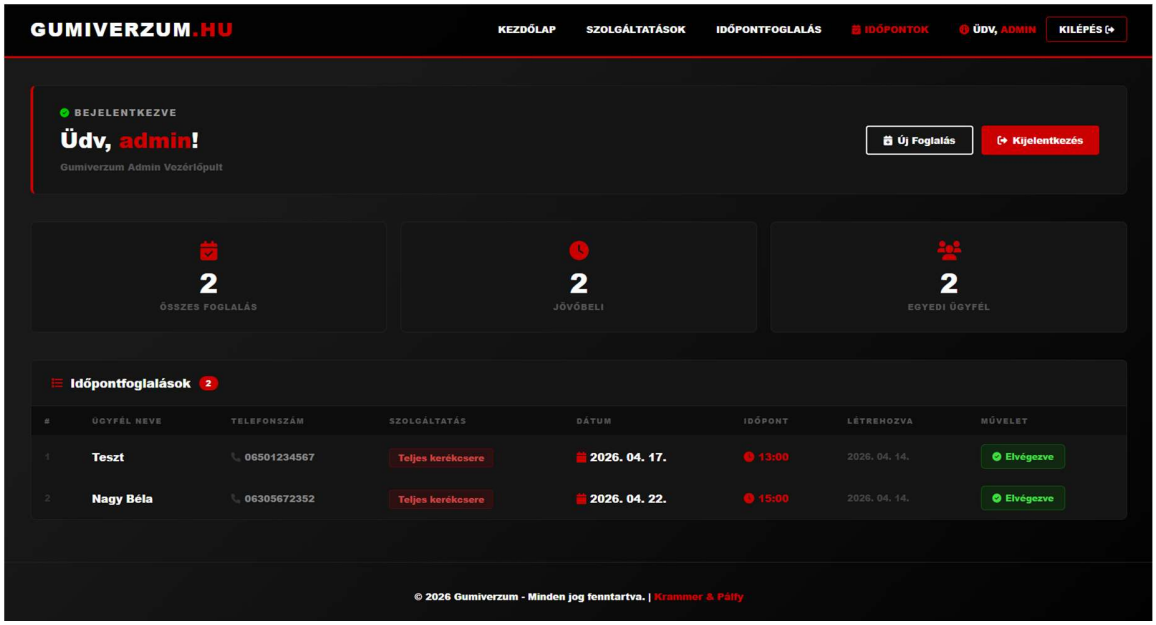
Kattintson a Belépés gombra!



A Vezérlőpult használata:

Amint belépett, egy teljesen új világ tárul Ön elé, a Dashboard. Itt nincsenek felesleges reklámok, csak a tiszta adatok.

- **Statisztika dobozok:** Az oldal tetején három doboz mutatja az aktuális számokat. Hány nyitott feladat van összesen? Hány feladat várható a mai naptól kezdve? Hány különböző ügyfél szerepel a rendszerben?

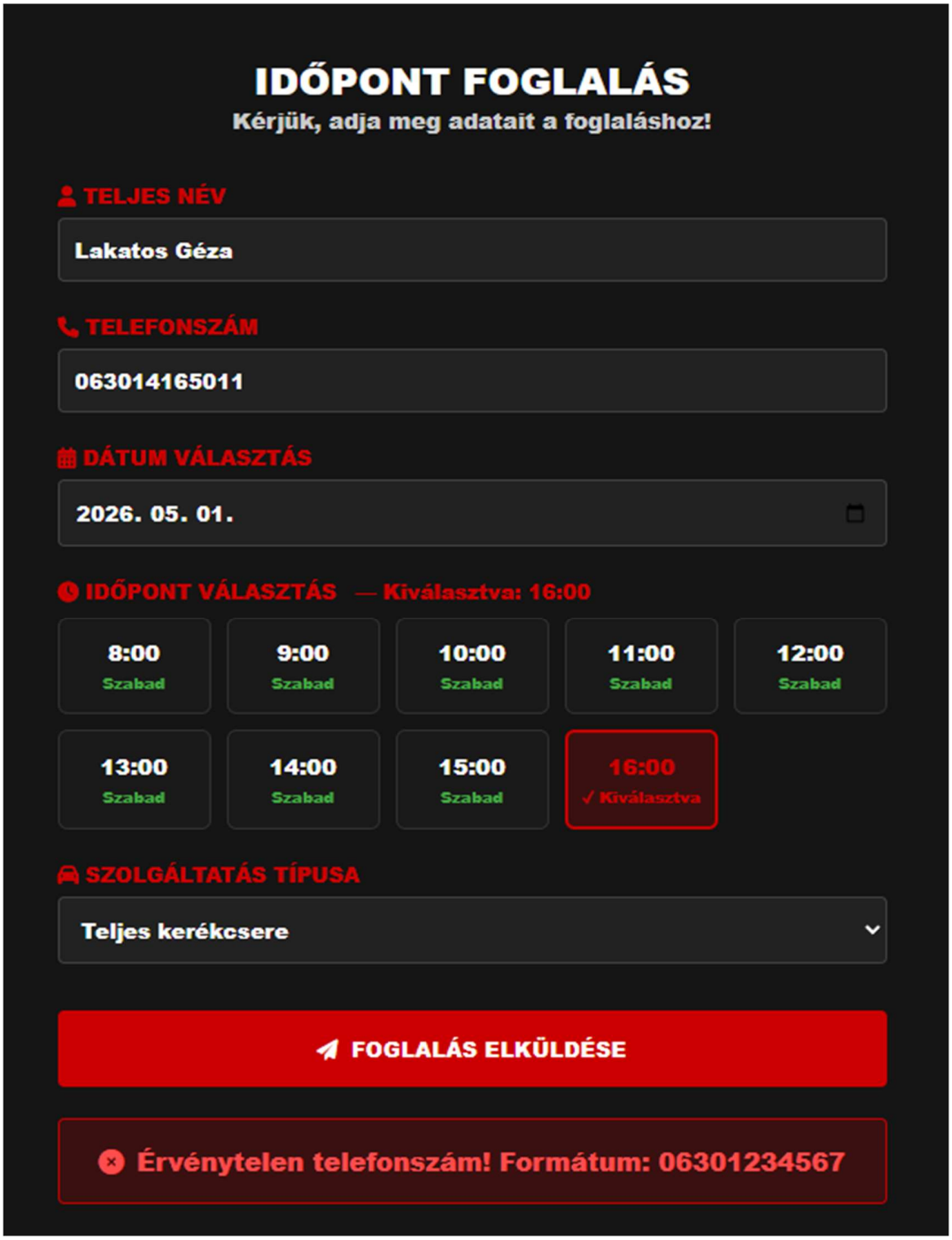


- **A Munkák Táblázata:** Egy nagy, áttekinthető listát lát. Minden sor egy-egy lefoglalt munkát jelent. Látja a nevet, a telefonszámot, a szolgáltatást, és a legfontosabbat: a Dátumot és az Időpontot.
- **A munka lezárása:** Miután az autót megemelték, a kerekeket lecserélték, és az ügyfél kifizette a számlát elégedetten távozva, Önnek csak egyetlen dolga van. Menjen a táblázatban az adott sor végére, és kattintson a **zöld "Elvégezve" gombra!** A sor azonnal, gyönyörűen eltűnik a képernyőről, az adatbázisban pedig a munka archiválódik.

Ha hiba történik... (Hibajelzések)

Előfordulhat, hogy valami nem sikerül elsőre. Olyankor a rendszer a képernyőn azonnal, piros betűkkel üzen Önnek. Ilyeneket írhat ki:

- **"Kérjük töltse ki az összes mezőt!"** -> Elfelejtett beírni egy adatot az űrlapon.
- **"Érvénytelen telefonszám! Formátum: 06301234567"** -> Rossz formátumban adta meg a számát.



"A(z) 10:00 órai időpont időközben betelt!" -> Extrém ritka eset! Miközben Ön lassan gépelte a nevét, egy másik ember a városban pont ezt az órát foglalta le a gépén előbb. Válasszon egy másik zöld gombot!

- **"Hibás felhasználónév vagy jelszó"** -> Elrontotta a belépési jelszót az admin felületen.

Bízunk benne, hogy ez a részletes útmutató minden kérdésére választ adott. Jó böngészést, könnyű foglalást és biztonságos autózást kívánunk!

Összefoglalás és Köszönetnyilvánítás

Projektértékelés és Szakmai tapasztalatok

Saját megítélésünk szerint az 1. fejezetben megfogalmazott szakmai célokat maradéktalanul sikerült elérni. A projekt lezárásakor büszkén értékelem az elkészült munkát, ugyanis a záródolgozat keretében nem csupán egy izolált kódblokkot, hanem egy teljes, iparági sztenderdeknek megfelelő Full-Stack webalkalmazást alkottam meg.

A rendszer nemcsak a szoftverfejlesztési és adatbázis-tervezési követelményeknek (3. normálforma, SQL Unique Constraints) felel meg, de a kliensoldali renderelés (Next.js) és az aszinkron API adatkezelés (Express.js) révén a modern elvárásokat is képviseli.

A legnagyobb szakmai kihívást a "versenyhelyzetek" és a dupla foglalások adatbázis-szintű kivédése jelentette. Sokat tanultam a relációs adatbázisok optimalizálásáról, az SQL lekérdezésekről, a biztonságos bejelentkezésről (JWT), valamint a kliens-szerver kommunikáció finomhangolásáról. A Tailwind CSS alkalmazása pedig rávilágított arra, hogyan lehet hatékonyan és villámgyorsan reszponzív, minden kijelzőn jól mutató felületeket kódolni egy komponens-alapú keretrendszerben. A program stabilan fut, a felhasználói felület intuitív, az adminisztrációs folyamat pedig jelentősen gyorsítja a szerviz mindennapi adminisztrációját.

A Gumiverzum utóélete, hasznosulása és Fejlesztési lehetőségek

A vizsgától függetlenül határozott meggyőződésem, hogy a szoftver komoly gyakorlati értékkel bír. A KKV szektorban (Kisebb gumiszervizek) óriási az igény az ilyen "dobozos", azonnal bevethető, mégis olcsó fenntartású megoldásokra. Minimális arculati testreszabás és egy felhőszolgáltatóhoz (VPS) történő élesítés után az alkalmazás azonnal használatba vehető bármely autószerviz által.

A jövőre nézve a szoftver számos ponton bővíthető:

1. **Automatikus E-mail/SMS visszaigazolás:** Külső API-k (pl. Twilio, Nodemailer) integrálásával a rendszer automatikus megerősítő üzenetet küldhetne az ügyfeleknek, illetve egy nappal a szerelés előtt emlékeztetőt is.
2. **Vizuális naptár nézet az Adminnak:** A jelenlegi táblázat helyett egy heti/havi interaktív naptárkomponens sokkal korszerűbbé tenné a műhely kapacitástervezését.

3. **Teljes körű gumiabroncs webshop:** A legfontosabb üzleti bővítés! A jelenlegi bemutató jellegű abroncskínálat továbbfejleszthető egy teljes értékű e-kereskedelmi modullá. Így a vásárlók nemcsak böngészhetnének, hanem kosárba is tehetnék a kiválasztott gumiabroncsokat, és egy online fizetési rendszeren (pl. SimplePay, Stripe) keresztül azonnal meg is vásárolhatnák azokat az időpontfoglalással egybekötve. A bevételek így hatványozottan nőhetnének.

Ezekkel a fejlesztésekkel az alkalmazást akár, havidíjas rendszerben is értékesíthető termékké lehet alakítani.

Köszönetnyilvánítás

Köszönetet szeretnénk nyilvánítani a felkészítő illetve szakmai tanáraink/oktatóinknak.

Őszinte köszönet és ezer hála tartozik Szalainé Török Edit tanárnőnek, akinek határtalan türelme és szakmai tudása nélkül ez a projekt nem jöhetett volna létre. A program alapjaiban is neki köszönheti a létezését.

Hálával tartozunk Méhes József tanár úrnak, mivel reggeli órákban is mindig készen állt bármilyen szakmai kérdésre, és mindenre azonnali, érthető és profi választ adott.

Külön köszönet jár Csukárdi Rajmund tanár úrnak, aki egy új világot nyitott meg előttünk a Next.js és Node.js bemutatásával, ami később ennek a projektnek is az alapját képezte.

Irodalomjegyzék

A projekttervezés, a frontend és backend fejlesztés, valamint az adatbázis-optimalizálás során az alábbi szakmai forrásokra támaszkodtam:

1. **Next.js Hivatalos Dokumentáció (React Framework)**
   Teljes URL: https://nextjs.org/docs/app
2. **Express.js Hivatalos Dokumentáció (Node.js backend)**
   Teljes URL: https://expressjs.com/en/4x/api.html
3. **Microsoft SQL Server dokumentáció – T-SQL referencia**
   Teljes URL: https://learn.microsoft.com/hu-hu/sql/t-sql/language-reference
4. **Tailwind CSS Hivatalos Dokumentáció (Utility-first CSS framework)**
   Teljes URL: https://tailwindcss.com/docs/installation
5. **JSON Web Token (JWT) Hivatalos Ismertető és Debugger**
   Teljes URL: https://jwt.io/introduction